

Task 1: To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

volatile sig_atomic_t signal_count = 0;

void signal_handler(int sig) {
    signal_count++;

    if (sig == SIGINT) {
        printf("\nSIGINT received. Count = %d\n", signal_count);
    }
    else if (sig == SIGTERM) {
        printf("\nSIGTERM received. Exiting...\n");
        exit(0);
    }
}

int main() {
    struct sigaction sa;

    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
```

```
GNU nano 7.2 task1.c *
sa.sa_handler = signal_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;

if (sigaction(SIGINT, &sa, NULL) == -1) {
    perror("sigaction SIGINT");
    return 1;
}

if (sigaction(SIGTERM, &sa, NULL) == -1) {
    perror("sigaction SIGTERM");
    return 1;
}

printf("Process PID: %d\n", getpid());
printf("Signal handlers registered.\n");
printf("Press Ctrl+C to generate SIGINT.\n");

while (1) {
    printf("Program running...\n");
    sleep(2);
}

return 0;
}
```

```

shreyas@Grigori:~$ mkdir -p ~/signal_handler_tasks
cd ~/signal_handler_tasks
shreyas@Grigori:~/signal_handler_tasks$ nano task1.c
shreyas@Grigori:~/signal_handler_tasks$ gcc task1.c -o task1
shreyas@Grigori:~/signal_handler_tasks$ ./task1
Process PID: 1169
Signal handlers registered.
Press Ctrl+C to generate SIGINT.
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
^C
SIGINT received. Count = 1
Program running...
Program running...
^C
SIGINT received. Count = 2
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...

```

```

shreyas@Grigori:~$ pgrep task1
1169
shreyas@Grigori:~$ kill -SIGTERM 1169
shreyas@Grigori:~$ kill -l
 1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
 6) SIGABRT     7) SIGBUS     8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD    18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU    23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF    28) SIGWINCH    29) SIGIO        30) SIGPWR
31) SIGSYS      34) SIGRTMIN   35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
shreyas@Grigori:~$ rm -f task1
shreyas@Grigori:~$ |

```

```
Program running...
Program running...
Program running...
Program running...
Program running...

SIGTERM received. Exiting...
shreyas@Grigori:~/signal_handler_tasks$ |
```

Task 2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

```
GNU nano 7.2 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child_pid = -1;

void sigint_handler(int sig) {
    printf("\nShell received SIGINT.\n");

    if (child_pid > 0) {
        printf("Forwarding SIGINT to foreground job: %d\n",
            child_pid);

        kill(child_pid, SIGINT);
    }
}

int main() {
    struct sigaction sa;
    int status;

    sa.sa_handler = sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
```

```

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("Mini shell PID: %d\n", getpid());
    printf("Shell SIGINT handler installed.\n");

    child_pid = fork();

    if (child_pid < 0) {
        perror("fork");
        return 1;
    }

    if (child_pid == 0) {
        printf("Foreground job started.\n");
        printf("Child PID: %d\n", getpid());

        signal(SIGINT, SIG_DFL);

        while (1) {
            printf("Foreground job running...\n");
            sleep(2);
        }
    }

```

```

shreyas@Grigori:~/signal_handler_tasks$ nano task2.c
shreyas@Grigori:~/signal_handler_tasks$ gcc task2.c -o task2
shreyas@Grigori:~/signal_handler_tasks$ ./task2
Mini shell PID: 1201
Shell SIGINT handler installed.
Foreground job PID: 1202
Press Ctrl+C to interrupt the foreground job.
Foreground job started.
Child PID: 1202
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
^C
Shell received SIGINT.
Forwarding SIGINT to foreground job: 1202
Foreground job exited with status 0
Job state updated. No foreground job active.
shreyas@Grigori:~/signal_handler_tasks$ echo "Shell is still active"

```

